UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Probleme de cautare si agenti adversariali

Inteligenta Artificiala

Student: Szarka Francesca
Grupa: 30234

Facultatea de Automatica
si Calculatoare

4 Decembrie 2025

Cuprins

1 Pacman — Project 1: Search **2**
1.1 Q1 — Depth First Search (DFS) 2
1.2 Q2 — Breadth First Search (BFS) 2
1.3 Q3 — Uniform Cost Search (UCS) 2
1.4 Q4 — A* Search 3
1.5 Q5 — Finding All the Corners 3
1.6 Q6 — Corners Problem: Heuristic 4
1.7 Q7 — Eating All The Dots 5
1.8 Q8 — Suboptimal Search 5

2 Pacman — Project 2: Multi-Agent Search **6**
2.1 Q1 — Reflex Agent 6
2.2 Q2 — Minimax Agent 7
2.3 Q3 — Alpha-Beta Agent 8

3 Pacman — Problema originala - Pacman Courier **9**

1 Pacman — Project 1: Search

1.1 Q1 — Depth First Search (DFS)

Algoritmul exploreaza un graf mergand cat mai adanc pe o cale inainte de a reveni pentru a explora si alte cai. Algoritmul foloseste o stiva (LIFO).

Vizitez un nod doar daca nu a mai fost vizitat - evit ciclurile. Stochez perechea (state, path) astfel incat sa reconstruiesc drumul.

```
def depthFirstSearch(problem: SearchProblem):
    stack = util.Stack()
    stack.push((problem.getStartState(), []))
    visited=set()
    while not stack.isEmpty():
        state, path = stack.pop()
        if problem.isGoalState(state):
            return path
        if state not in visited:
            visited.add(state)
            for successor, action, cost in problem.getSuccessors(state):
                if successor not in visited:
                    stack.push((successor, path + [action]))
    return []
```

1.2 Q2 — Breadth First Search (BFS)

Algorimtul exploreaza un graf mergand prima data la vecinii unui nod inainte sa treaca la nivelul urmator. Foloseste o coada (FIFO).

Returneaza mereu drumul cel mai scurt in graf. Aceeasi structura (state,path), dar ordinea explorarii e diferita.

```
def breadthFirstSearch(problem: SearchProblem):
    queue = util.Queue()
    queue.push((problem.getStartState(), []))
    visited = set()
    while not queue.isEmpty():
        state, path = queue.pop()
        if problem.isGoalState(state):
            return path
        if state not in visited:
            visited.add(state)
            for successor, action, cost in problem.getSuccessors(state):
                if successor not in visited:
                    queue.push((successor, path + [action]))
    return []
```

1.3 Q3 — Uniform Cost Search (UCS)

Foloseste PriorityQueue - prioritate = cost total acumulat. UCS garanteaza cel mai mic cost.

Spre deosebire de DFS/BFS, tine cont de cost la fiecare pas.

```
def uniformCostSearch(problem: SearchProblem):
```

```
    pq = util.PriorityQueue()
    pq.push((problem.getStartState(), [], 0), 0)
    visited = set()
    while not pq.isEmpty():
        state, path, cost = pq.pop()
        if problem.isGoalState(state):
            return path
        if state not in visited:
            visited.add(state)
            for successor, action, stepCost in problem.getSuccessors(
                state):
                if successor not in visited:
                    newCost = cost + stepCost
                    newPath = path + [action]
                    pq.push((successor, newPath, newCost), newCost)
    return []
```

1.4 Q4 — A* Search

Extinde UCS introducand euristica. Prioritate = cost + heuristic.

Combin costul deja parcurs cu o estimare h(n) a distantei ramase.

Daca h(n)=0, A* se comporta ca UCS.

Functia prioritara: f(n)=g(n)+h(n).

```
def aStarSearch(problem: SearchProblem, heuristic=nullHeuristic):
    pq=util.PriorityQueue()
    pq.push((problem.getStartState(),[],0),0)
    visited=set()
    while not pq.isEmpty():
        state,path,cost=pq.pop()
        if problem.isGoalState(state):
            return path
        if state not in visited:
            visited.add(state)
            for successor, action, stepCost in problem.getSuccessors(
                state):
                if successor not in visited:
                    newCost = cost + stepCost
                    newPath = path + [action]
                    priority = newCost + heuristic(successor, problem)
                    pq.push((successor,  newPath, newCost), priority)

    return []
```

1.5 Q5 — Finding All the Corners

Scopul este sa gasim cel mai scurt traseu care trece prin toate colturile hartii.

Starea este reprezentata minimal ca:

$$(state) = (pacmanPosition,\ visitedCorners)$$

La inceput niciun colt nu este vizitat.

```python
def getStartState(self):
    return (self.startingPosition, ())
```

Succesorii sunt generati prin extinderea nodului cu 4 miscari posibile (N,S,E,W), marcand colturi daca sunt atinse.

```python
def getSuccessors(self, state: Any):
    position, visitedCorners = state
    x, y = position
    dx, dy = Actions.directionToVector(action)
    nextx, nexty = int(x + dx), int(y + dy)
    if not self.walls[nextx][nexty]:
        nextPosition = (nextx, nexty)
        newVisitedCorners = visitedCorners
        if nextPosition in self.corners and nextPosition not in visitedCorners:
            newVisitedCorners = tuple(sorted(visitedCorners + (nextPosition,)))
            nextState = (nextPosition, newVisitedCorners)
            successors.append( (nextState, action, 1) )
        self._expanded += 1
        return successors
```

In `getSuccessors` verific fiecare miscare posibila: daca pozitia urmatoare nu este perete, o adaug ca succesor. Daca pozitia este un colt si nu a fost vizitat inca, il introduc in tuplul `visitedCorners` si creez starea actualizata.

Scopul este atins cand toate cele 4 colturi sunt vizitate:

```python
def isGoalState(self, state: Any):
    position,visitedCorners = state
    return len(visitedCorners) == 4
```

1.6 Q6 — Corners Problem: Heuristic

Cerinta este implementarea unei euristici pentru CornersProblem.

-distanta minima = distanta pana la cel mai apropiat colt nevizitat.

-distanta maxima = cea mai mare distanta dintre doua colturi ramase.

Aceasta euristica verifica mai intai ce colturi nu au fost vizitate inca. Apoi calculeaza distanta Manhattan de la pozitia curenta pana la cel mai apropiat colt nevizitat si o retine ca `minDistanceToCorner`. In continuare, parcurge toate perechile de colturi nevizitate si gaseste distanta maxima dintre ele, stocata in `maxDistanceBetweenCorners`. La final returneaza suma celor doua valori, care reprezinta estimarea folosita de A*.

```python
def cornersHeuristic(state: Any, problem: CornersProblem):
    corners = problem.corners
    position, visitedCorners = state
    unvisitedCorners = [corner for corner in corners if corner not in visitedCorners]
    if not unvisitedCorners:
        return 0
    distancesToCorners = [util.manhattanDistance(position, corner) for corner in unvisitedCorners]
    minDstanceToCorner = min(distancesToCorners)
```

```
    maxDistanceBetweenCorners = 0
    for i in range(len(unvisitedCorners)):
        for j in range(i+1, len(unvisitedCorners)):
            distance = util.manhattanDistance(unvisitedCorners[i],
                unvisitedCorners[j])
            if distance > maxDistanceBetweenCorners:
                maxDistanceBetweenCorners = distance
    return minDstanceToCorner + maxDistanceBetweenCorners
```

1.7 Q7 — Eating All The Dots

Aici definim euristica pentru `FoodSearchProblem`. Pacman trebuie sa manance toate bucatile de mancare de pe harta cu cat mai putine miscari. Heuristica returneaza distanta `mazeDistance` pana la cel mai indepartat punct de mancare.

```
def foodHeuristic(state: Tuple[Tuple, List[List]], problem:
    FoodSearchProblem):
    position, foodGrid = state
    foodList = foodGrid.asList()
    if not foodList:
        return 0
    if 'distances' not in problem.heuristicInfo:
        problem.heuristicInfo['distances'] = {}
    maxDistance = 0
    for food in foodList:
        key = (position, food)
        if key not in problem.heuristicInfo['distances']:
            distance = mazeDistance(position, food, problem.
                startingGameState)
            problem.heuristicInfo['distances'][key] = distance
        else:
            distance = problem.heuristicInfo['distances'][key]
        if distance > maxDistance:
            maxDistance = distance
    return maxDistance
```

In `foodHeuristic` extrag pozitia lui Pacman si lista cu mancarea ramasa. Daca nu mai exista mancare, returnez 0. Pentru fiecare punct de mancare calculez distanta reala `mazeDistance` dintre Pacman si acel punct. Rezultatele sunt memorate in `heuristicInfo` pentru a nu recalcua distanta de mai multe ori. Parcurg lista si pastrez doar distanta cea mai mare, deoarece aceea reprezinta drumul minim garantat pe care Pacman trebuie sa il faca. Functia returneaza aceasta valoare ca estimare euristica.

1.8 Q8 — Suboptimal Search

Aici completam un agent care mananca suboptimal cele mai apropiate bucati de mancare pe rand (greedy).

Goal state pentru prima mancare gasita:

```
def isGoalState(self, state):
    x,y = state
    return self.food[x][y]
```

Algoritmul folosit pentru gasirea drumului pana la cea mai apropiata bucata de mancare este BFS:

```
def findPathToClosestDot(self, gameState: pacman.GameState):
    problem = AnyFoodSearchProblem(gameState)
    return search.bfs(problem)
```

2 Pacman — Project 2: Multi-Agent Search

2.1 Q1 — Reflex Agent

Agentul a fost modificat astfel incat decizia sa tina cont de distanta fata de mancare, pozitia fantomelor, starea lor *scared* si penalizarea pentru oprire. Functia returneaza un scor numeric pe baza starii urmatoare simulate prin `generatePacmanSuccessor`.

```
def evaluationFunction(self, currentGameState: GameState, action):
    successorGameState = currentGameState.generatePacmanSuccessor(action
        )
    newPos = successorGameState.getPacmanPosition()
    newFood = successorGameState.getFood()
    newGhostStates = successorGameState.getGhostStates()
    newScaredTimes = [ghostState.scaredTimer for ghostState in
        newGhostStates]
    foodList = newFood.asList()
        score = successorGameState.getScore()
        if foodList:
            distances = [manhattanDistance(newPos, food) for food in
                foodList]
            score += 1.5 / min(distances)
        for ghost, scaredTime in zip(newGhostStates, newScaredTimes):
            ghostPos = ghost.getPosition()
            dist = manhattanDistance(newPos, ghostPos)
            if scaredTime > 0:
                score += 2.0 / max(1, dist)
            else:
                if dist <= 1:
                    score -= 500
                else:
                    score -= 2.0 / dist
        if action == Directions.STOP:
            score -= 10
        return score
```

Se simuleaza actiunea, iar scorul creste cand Pacman se apropie de mancare si scade cand se apropie de fantome. STOP este penalizat. Actiunea finala este cea cu scor maxim.

Evaluarea foloseste distanta Manhattan pentru a decide cat de buna este o mutare: cu cat o bucata de mancare este mai aproape, cu atat scorul creste mai mult. In acelasi timp, se verifica pozitiile fantomelor — daca sunt aproape si nu sunt speriate, scorul scade drastic, ceea ce impinge agentul sa fuga. In schimb, daca fantomele sunt speriate, agentul este incurajat sa se apropie, deoarece le poate manca.

2.2 Q2 — Minimax Agent

Am implementat agentul Minimax, capabil sa joace impotriva oricarui numar de fantome. Pacman este nod MAX, fantomele noduri MIN. Se cauta recursiv pana la adancimea setata.

```python
def getAction(self, gameState: GameState):
    numAgents = gameState.getNumAgents()
    def minimax(state, agentIndex, depth):
        if depth == self.depth or state.isWin() or state.isLose():
            return self.evaluationFunction(state)
        legalActions = state.getLegalActions(agentIndex)
        if not legalActions:
            return self.evaluationFunction(state)
        def next_agent_depth(agentIndex, depth):
            nextAgent = (agentIndex + 1) % numAgents
            nextDepth = depth + 1 if nextAgent == 0 else depth
            return nextAgent, nextDepth
        if agentIndex == 0:
            # Pacman
            value = float('-inf')
            for action in legalActions:
                successor = state.generateSuccessor(agentIndex, action)
                nextAgent, nextDepth = next_agent_depth(agentIndex,
                    depth)
                value = max(value, minimax(successor, nextAgent,
                    nextDepth))
            return value
        else:
            # Ghost
            value = float('inf')
            for action in legalActions:
                successor = state.generateSuccessor(agentIndex, action)
                nextAgent, nextDepth = next_agent_depth(agentIndex,
                    depth)
                value = min(value, minimax(successor, nextAgent,
                    nextDepth))
            return value
    bestAction = None
    bestValue = float('-inf')
    legalActions = gameState.getLegalActions(0)
    for action in legalActions:
        successor = gameState.generateSuccessor(0, action)
        nextAgent = 1 % numAgents
        depth = 0
        if nextAgent == 0:
            depth = 1
        value = minimax(successor, nextAgent, depth)
        if value > bestValue or bestAction is None:
             bestValue = value
             bestAction = action
    return bestAction
```

Pacman maximizeaza scorul, fantomele il minimizeaza. Dupa fiecare trecere prin ultimul agent creste adancimea. Se opreste la depth sau in stari terminale.

Functia `minimax` primeste starea curenta, agentul care muta si nivelul de adancime. Pentru Pacman se initializeaza valoarea cu -inf si se alege maximul dintre succesorii posibili, iar pentru fantome se initializeaza cu +inf si se ia minimul. Trecerea la urmatorul agent se face modular, astfel incat dupa ultima fantoma ciclul revine la Pacman. La nivelul radacina (`getAction`), sunt evaluate toate mutarile posibile ale lui Pacman, iar actiunea cu valoarea Minimax cea mai mare este returnata drept mutare optima.

2.3 Q3 — Alpha-Beta Agent

Implementare Minimax optimizata prin Alpha-Beta pruning. Ramurile care nu pot influenta rezultatul sunt ignorate, reducand timpul de cautare.

```python
def getAction(self, gameState: GameState):
    numAgents = gameState.getNumAgents()
    def alphabeta(state, agentIndex, depth, alpha, beta):
        if depth == self.depth or state.isWin() or state.isLose():
            return self.evaluationFunction(state)
        legalActions = state.getLegalActions(agentIndex)
        if not legalActions:
            return self.evaluationFunction(state)
        def next_agent_depth(agentIndex, depth):
            nextAgent = (agentIndex + 1) % numAgents
            nextDepth = depth + 1 if nextAgent == 0 else depth
            return nextAgent, nextDepth
        # max node: Pacman
        if agentIndex == 0:
            value = float('-inf')
            for action in legalActions:
                successor = state.generateSuccessor(agentIndex, action)
                nextAgent, nextDepth = next_agent_depth(agentIndex, depth)
                value = max(value, alphabeta(successor, nextAgent, nextDepth, alpha, beta))
                if value > beta:
                    return value
                alpha = max(alpha, value)
            return value
        # min node: Ghost
        else:
            value = float('inf')
            for action in legalActions:
                successor = state.generateSuccessor(agentIndex, action)
                nextAgent, nextDepth = next_agent_depth(agentIndex, depth)
                value = min(value, alphabeta(successor, nextAgent, nextDepth, alpha, beta))
                if value < alpha:
                    return value
                beta = min(beta, value)
            return value
    bestValue = float('-inf')
    bestAction = None
    alpha = float('-inf')
    beta = float('inf')
```

```
    legalActions = gameState.getLegalActions(0)
    for action in legalActions:
        successor = gameState.generateSuccessor(0, action)
        nextAgent = 1 % numAgents
        depth = 0
        if nextAgent == 0:
            depth = 1
        value = alphabeta(successor, nextAgent, depth, alpha, beta)
        if value > bestValue or bestAction is None:
            bestValue = value
            bestAction = action
        alpha = max(alpha, bestValue)
    return bestAction
```

Functia `alphabeta` este aproape identica cu Minimax, dar acum mentine doua variabile: alfa pentru cel mai bun scor gasit de MAX si beta pentru cel mai bun scor gasit de MIN. La nodurile Pacman (MAX) se actualizeaza alfa, iar la nodurile fantomelor (MIN) se actualizeaza beta. Daca in orice punct alfa mai mare sau egal decat beta, ramura curenta se elimina deoarece nu poate produce un rezultat mai bun pentru agentul aflat la decizie.

La radacina, agentul parcurge toate mutarile posibile ale lui Pacman, executand apelul `alphabeta` pentru fiecare si retinand actiunea cu scorul final cel mai ridicat. Astfel, obtinem acelasi rezultat ca Minimax, dar cu explorare mult mai eficienta si un numar redus de stari generate.

3 Pacman — Problema originala - Pacman Courier

Problema Courier extinde mecanicile clasice ale proiectului Pacman, transformand agentul dintr-un simplu colector de puncte intr-un curier care trebuie sa parcurga o harta, sa ridice o serie de livrari plasate in pozitii fixe, iar ulterior sa se intoarca in punctul de plecare. In aceasta varianta nu luam in considerare prezenta fantomelor, accentul fiind pus pe optimizarea traseului si reducerea numarului total de pasi.

Problema poate fi privita ca o variatie simplificata a Traveling Salesman Problem, insa cu o constrangere suplimentara: ordinea livrarilor nu este aleasa liber, ci este impusa de indexul curent al coletului. Astfel, Pacman nu poate ridica livrarile in orice ordine, ci trebuie sa respecte un traseu secvential, ceea ce transforma problema intr-un spatiu de stari bine definit si determinist. Aceasta structurare permite aplicarea algoritmilor de cautare precum BFS, UCS sau A*, pentru a obtine traseul optim.

Clasa `CourierProblem` mosteneste `SearchProblem` si ofera implementarea completa a spatiului de cautare necesar agentilor de tip BFS, UCS, A*. Structura este clara, modulara si defineste explicit fiecare componenta necesara unei probleme de cautare:

- **starea initiala**
- **functia succesor**
- **functia de testare a starii scop**
- **functia de cost**

La initializare, clasa primeste un `GameState`, din care extrage toate elementele relevante pentru problema:

1. **peretii hartii** (`walls`) — restrictii de miscare
2. **pozitia de start a lui Pacman** — prima coordonata a starii
3. **capsulele (livrarile)** — extrase cu `getCapsules()`, sortate pentru consistenta

4. **pozitia finala (exit)** = pozitia de start
5. **starea initiala** $(pos, 0)$ — Pacman + indexul primei livrari

Livrarile sunt memorate ca tuplu ordonat. Aceasta garanteaza ca doua stari identice vor avea aceeasi semnatura, lucru esential pentru evitarea reexplorarii nodurilor cand folosesc UCS sau A*.

Fiecare stare este reprezentata ca:

$$state = (position,\ index)$$

unde:

- **position** – coordonatele actuale ale lui Pacman
- **index** – cate livrari au fost colectate pana in prezent

Indexul functioneaza ca un contor secvential. Daca Pacman ajunge exact pe locatia `deliveries[index]`, atunci indexul creste automat. Aceasta elimina nevoia unui set de livrari vizitate si reduce masiv spatiul de cautare.

Conditia de final este stricta:

$$index == len(deliveries)\ \wedge\ position == startPos$$

Cu alte cuvinte:

1. toate livrarile au fost ridicate
2. agentul s-a intors la locul initial

Generarea succesorilor

Functia `getSuccessors` exploreaza patru directii posibile: Nord, Sud, Est, Vest. Pentru fiecare mutare:

1. se verifica daca pozitia urmatoare nu este perete
2. daca Pacman atinge urmatorul colet din lista, indexul creste
3. succesorul se adauga cu cost 1

Aceasta functie defineste complet spatiul de cautare, iar algoritmul nu are nevoie de informatii suplimentare pentru a gasi solutia optima.

$$Cost(actions) = len(actions)$$

Fiecare pas costa exact 1, ceea ce face ca algoritmi precum BFS sau UCS sa returneze automat drumul cel mai scurt. In cazul A*, eficienta creste semnificativ daca se foloseste o euristica admisibila.

```
class CourierProblem(search.SearchProblem):

    Pacman Curier:
    Pacman trebuie sa colecteze toate capsulele (deliveries) , apoi sa
       se intoarca la pozitia de start.
    Nu iau in considerare fantome in aceasta varianta.


    def __init__(self, startingGameState: pacman.GameState):
        self.walls = startingGameState.getWalls()
        # pozitia initiala a lui Pacman -> aici va reveni la final
```

```python
        self.startPos = startingGameState.getPacmanPosition()
        # preiau capsulele (simbolizate prin 'o') si le sortam pentru a avea o ordine fixa
        # Ex ordonare:[(3,4),(5,2),(8,7)]
        self.deliveries = tuple(sorted(startingGameState.getCapsules()))
        # punctul final (exit) este locul de start
        self.exitPos = self.startPos
        # starea initiala a problemei:
        # (pozitia_curenta, index_delivery_curenta)
        # index=0 inseamna ca niciun colet nu a fost ridicat
        self.startState = (self.startPos, 0)
        # contor intern pentru statistici (nu trebuie modificat)
        self._expanded = 0

    def getStartState(self):
        return self.startState

    def isGoalState(self, state):
        pos, idx = state
        # scopul este atins doar daca:
        # 1. idx == numar total de livrari (toate capsulele luate)
        # 2. Pacman a revenit exact la startPos (exit)
        return idx == len(self.deliveries) and pos == self.exitPos

    def getSuccessors(self, state):
        successors = []
        (x, y), index = state
        # incrementez noduri expandate (statistica pentru afisare)
        self._expanded += 1
        for action in [Directions.NORTH, Directions.SOUTH, Directions.EAST, Directions.WEST]:
            dx, dy = Actions.directionToVector(action)
            # calculez noua pozitie
            nx, ny = int(x + dx), int(y + dy)
            # daca noua pozitie este un perete -> ignoram miscarea
            if self.walls[nx][ny]:
                continue
            # implicit Pacman ramane la aceeasi livrare curenta
            newIndex = index
            # daca Pacman se afla fix pe urmatoarea locatie de livrare -> trece la urmatoarea
            # ex: deliveries[index] = (5,2) -> Pacman trebuie sa ajunga acolo ca index sa creasca
            if index < len(self.deliveries) and (nx, ny) == self.deliveries[index]:
                newIndex += 1  # a colectat coletul
            successors.append((((nx, ny), newIndex), action, 1))
        return successors

    def getCostOfActions(self, actions):
        # daca nu exista actiuni -> cost mare (invalid)
        if actions is None:
            return 999999
        # pornesc din startPos si verific fiecare actiune
        x, y = self.startPos
```

```
        for a in actions:
            dx, dy = Actions.directionToVector(a)
            x, y = int(x + dx), int(y + dy)
            # daca vreodata se loveste de perete -> secventa invalida
            if self.walls[x][y]:
                return 999999
        # costul total este lungimea traseului (fiecare pas costa 1)
        return len(actions)
```